graphics systems, you imagine you have a turtle you can command to move forward n squares, turn left, and turn right. You can also make the turtle raise and lower a pen. If the pen is lowered, a line will be drawn tracing the turtle's subsequent movements. Very few of these turtles exist in the wild, so we tend to simulate them inside computers. To draw a set of three 5x5 squares, we might write the following:

```
3.times do
  forward(8)
  pen_down
  4.times do
    forward(4)
    left
  end
  pen_up
end
```

Yes, the forward(4) is correct in this code. The initial point is always drawn.

Clearly, pen_down, forward, left, and pen_up can be implemented as Ruby methods. However, this API has all the methods being called without a receivers. For that to work we either have to be within a class that defines those methods (or is a child of such a class) or we have to make the methods global. Neither of those allows for the API to be as simple as we have defined it here.

instance_eval to the rescue. We can define a class Turtle that defines the various methods we need as instance methods. We'll also define a walk method, which will execute our turtle DSL, and a draw method to draw the resulting picture:

```
class Turtle
  def left; ... end
  def right; ... end
  def forward(n); ... end
  def pen_up; .. end
  def pen_down; ... end
  def walk(...); end
  def draw; ... end
end
```

If we implement walk correctly, we can then write the code like this:

```
turtle = Turtle.new
turtle.walk do
  3.times do
    forward(8)
    pen_down
    4.times do
      forward(4)
      left
    end
    pen_up
  end
end
turtle.draw
```